**Michael Liebing / 18323016**
**Jonathan Otto / 18323017**

# EB3203 Pengolahan Citra Biomedis

## Tugas 1 - Improving Eigenface Recognition Rate

**SOAL**

Working with the Brute-Force Eigenface Recognition Mode:

1. Find the relation of eigenface number & recognition accuracy based on eigenfaces derived from subset 0, to images in subsets 0, 1 and 2, without any image pre-processing algorithm (determine the number of eigenface first, aim for at least 90% accuracy in subset 0).
2. Propose a solution (pre-processing algorithm) to overcome the problem of illumination sensitivity (without changing the number of eigenface from #1)
   a. Repeat #1 after adding image pre-processing algorithm only to images of subsets 1 and 2, prior to performing recognition
   b. Explain how the pre-processing algorithm work and compare at least two types of preprocessing algorithm
3. Explore the minimum image size & minimum number of eigenface required to maintain -2% and -5% accuracy from the accuracy you get on #1! (Example: if in #1 your best eigenface accuracy with pre-processing is 85%; then you need to find the minimum image size & minimum number of eigenface components that would give you 83% and 80% accuracy)
4. How would you find a good initial guess for a threshold value?
5. How would you modify face_recognition.m to implement the threshold based recognition? In your answer:
   - Please add th as an input argument representing threshold value
   - Specify which lines would you modify from the original code so that the output of face_recognition.m will represent best match based on threshold
   - Explain how the replacement code works
   - Include a work-around in your code to mitigate situations where there are multiple entries in the dataset that meet a given recognition threshold
6. How would you build a procedure to generate a FAR/FRR/EER?
   - Create a dedicated Matlab function where the output arguments are FAR, FRR, and EER values.
   - You may choose your own input arguments, please provide explanations.

**Michael Liebing / 18323016**
**Jonathan Otto / 18323017**

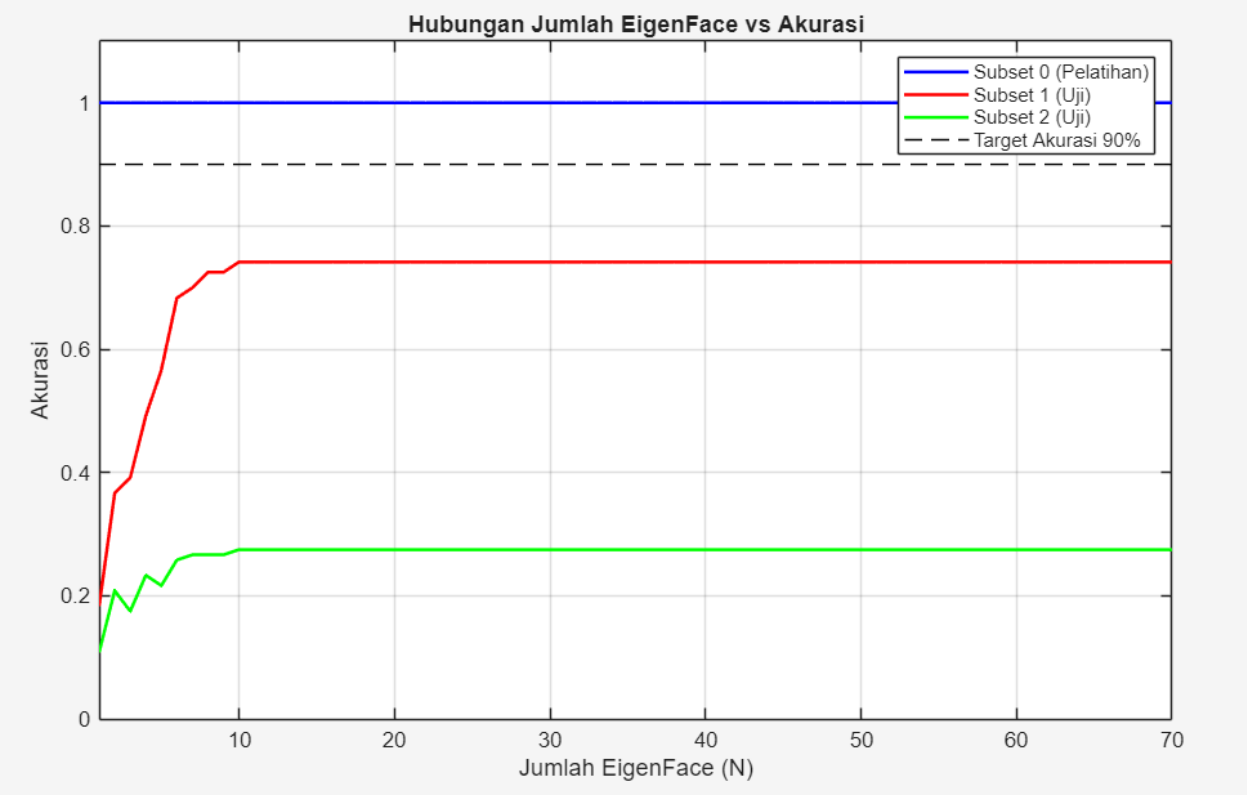
**JAWABAN**

Seluruh kode Matlab yang digunakan dalam pengerjaan tugas ini dapat diakses melalui *link* Github sebagai berikut,

https://github.com/BingBongCoder/EB3203_Kelompok7/tree/main/Tugas1

**Nomor 1**

Berikut adalah hasil pengujian korelasi tanpa *pre-processing* untuk subset0, subset1, subset2:




*Gambar 1 - Grafik Hubungan Jumlah Komponen EigenFace dan Akurasi dari Subset 0, 1, dan 2*

Berikut hasil tabulasi data akurasi terhadap jumlah Eigenface:

*Tabel 1 - Hasil Data Akurasi Korelasi Tanpa Pre-processing*

| Akurasi Subset | Jumlah Eigenface (N) | | | | | | Jumlah Eigenface Minimum untuk Stabil |
|---|---|---|---|---|---|---|---|
| | 1 | 5 | 10 | 20 | 40 | 70 | |
| Subset0 | 100.0% | 100.0% | 100.0% | 100.0% | 100.0% | 100.0% | 1 |
| Subset1 | 18.3% | 56.7% | 74.2% | 74.2% | 74.2% | 74.2% | 10 |
| Subset2 | 10.8% | 21.7% | 27.5% | 27.5% | 27.5% | 27.5% | 10 |

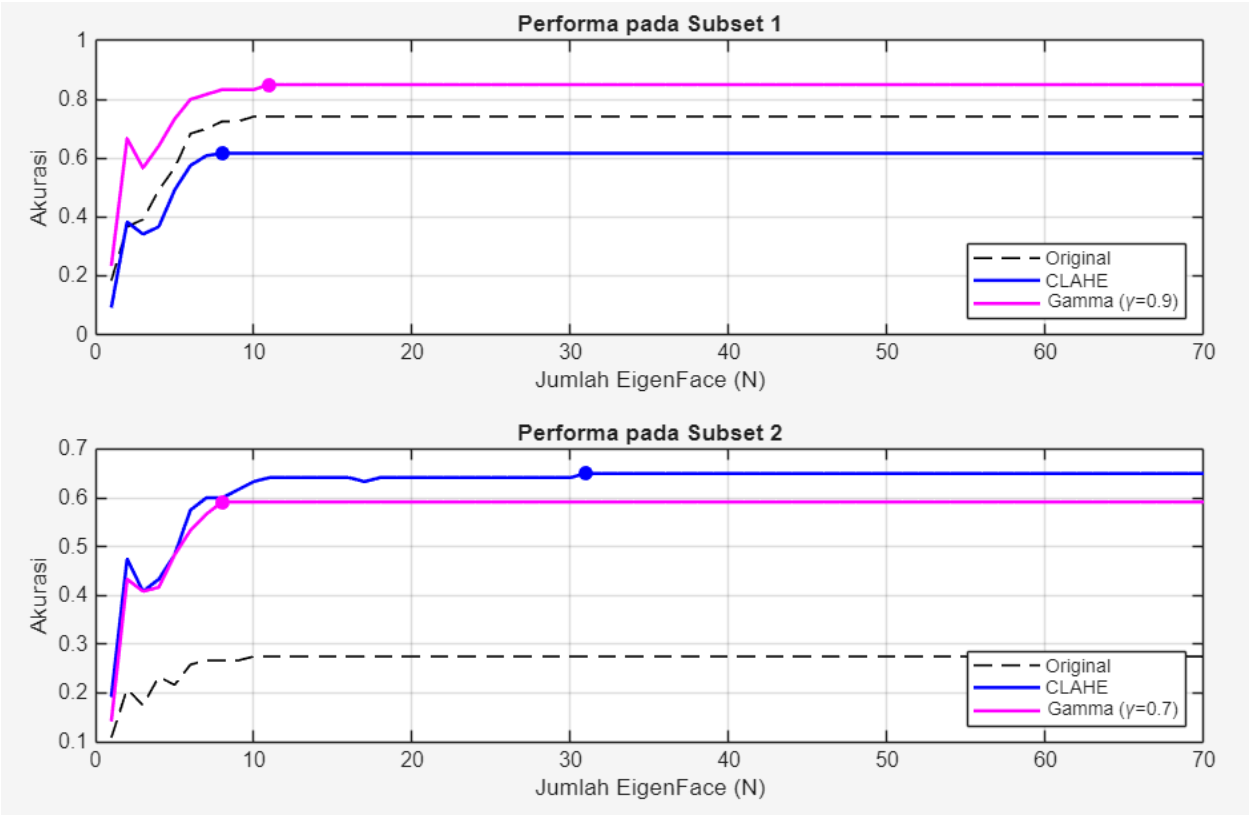
```
PENGERJAAN NOMOR 1
Jumlah N minimum untuk akurasi menjadi konstan:
 Subset 0: 1 (Akurasi 100.00%)
```

```
Subset 1: 10 (Akurasi 74.17%)
Subset 2: 10 (Akurasi 27.50%)
```

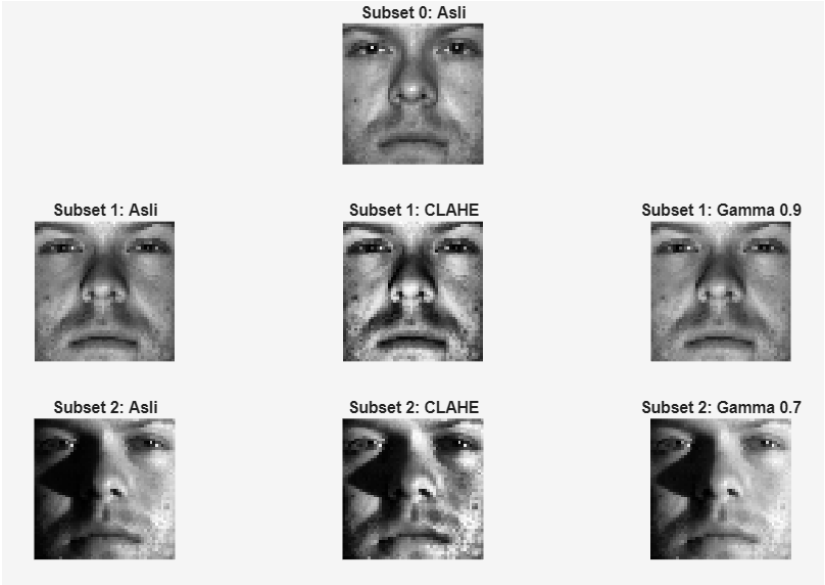
*Gambar 2 - Output Kode Matlab Pengerjaan Nomor 1*

## Nomor 2

a. Berikut hasil uji akurasi,




*Gambar 3 Grafik Hubungan Jumlah Komponen EigenFace dan Akurasi dari Subset 0, 1, dan 2 dengan Pre-processing*




*Gambar 4 - Hasil Pre-processing Tiap Metode untuk Subset1 dan Subset2*

*Tabel 2 - Hasil Akurasi CLAHE*

| Akurasi Subset CLAHE | Jumlah Eigenface (N) | | | | | | Jumlah Eigenface Minimum untuk Stabil |
|---|---|---|---|---|---|---|---|
| | 1 | 5 | 10 | 20 | 40 | 70 | |
| Subset1 | 9.17% | 49.17% | 61.67% | 61.67% | 61.67% | 61.67% | 8 |
| Subset2 | 19.17% | 48.33% | 63.33% | 64.17% | 65.00% | 65.00% | 31 |

*Tabel 3 - Hasil Akurasi Gamma Correction*

| Akurasi Subset Gamma | Jumlah Eigenface (N) | | | | | | Jumlah Eigenface Minimum untuk Stabil |
|---|---|---|---|---|---|---|---|
| | 1 | 5 | 10 | 20 | 40 | 70 | |
| Subset1 | 23.33% | 73.33% | 83.33% | 85.00% | 85.00% | 85.00% | 11 |
| Subset2 | 14.17% | 48.33% | 59.17% | 59.17% | 59.17% | 59.17% | 8 |

```
PENGERJAAN NOMOR 2

Akurasi Maksimal & N Minimum
 Subset 1 (CLAHE)    : 61.67% pada N minimum = 8
 Subset 1 (Gamma 0.9): 85.00% pada N minimum = 11

 Subset 2 (CLAHE)    : 65.00% pada N minimum = 31
 Subset 2 (Gamma 0.7): 59.17% pada N minimum = 8
```

*Gambar 5 - Output Kode Matlab Pengerjaan Nomor 2*

b. Cara kerja metode *pre-processing* yang digunakan adalah *Contrast Limited Adaptive Histogram Equalization* dan *Gamma Intensity Correction*. CLAHE merupakan perluasan dari metode *Histogram Equalization*. Perbedaan yang mencolok antara keduanya adalah CLAHE akan memotong citra menjadi sub-citra kecil dan dilakukan *Histogram Equalization* secara lokal pada setiap sub-citra tersebut. Metode ini membuat citra yang dihasilkan memiliki kontras yang

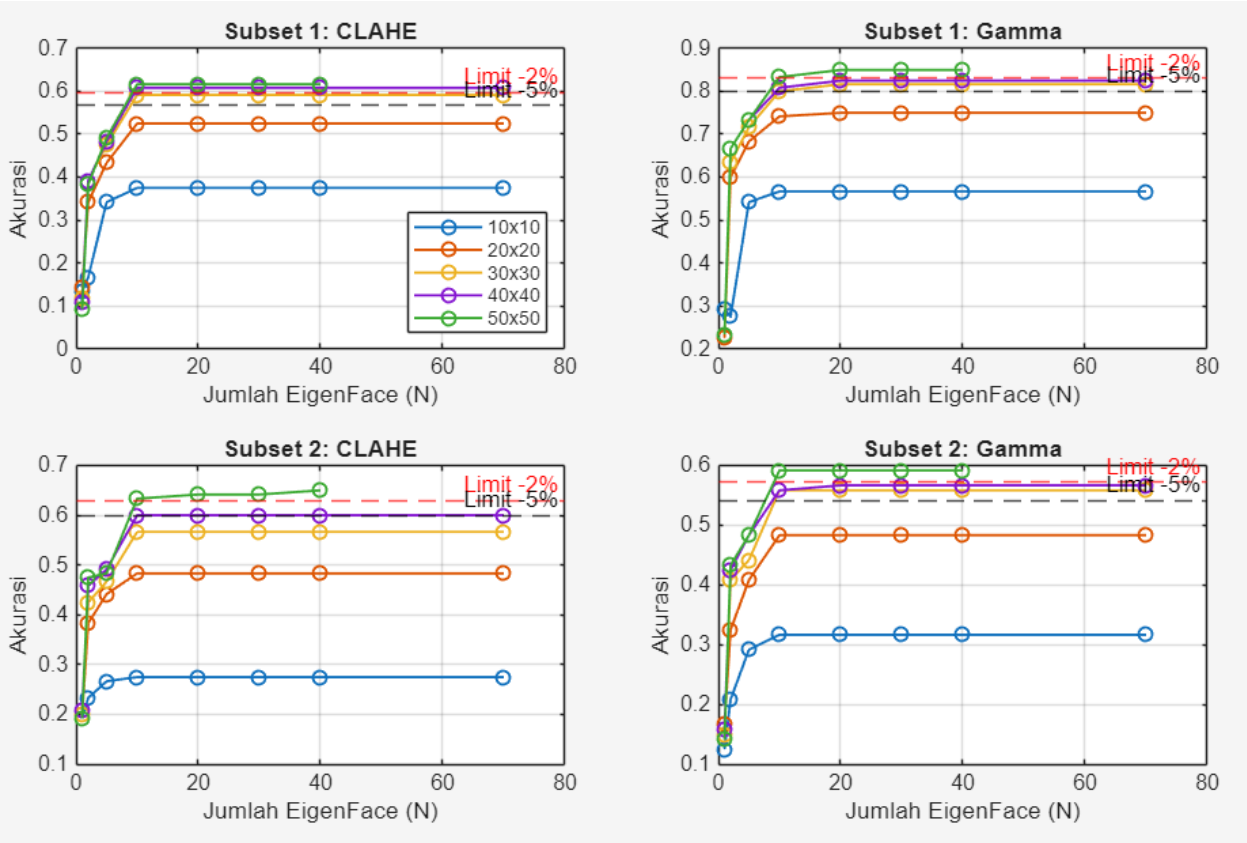
lebih natural. *Gamma Intensity Correction* merupakan sebuah operasi titik dengan mengubah nilai intensitas tiap pixel menjadi pangkat gamma.

$$I_{gam}(x, y) = I(x, y)^{\gamma}$$

Ketika $\gamma$ diatur menjadi > 1, daerah gelap pada citra asli menjadi lebih terang. Demikian pula, Ketika $\gamma$ diatur menjadi < 1, daerah terang pada citra asli menjadi lebih gelap.

**Nomor 3**

*Gambar 6 - Hasil Pengecilan Ukuran Gambar*




*Gambar 7 - Grafik Analisis Parameter Target -2% dan -5%*

```
PENGERJAAN NOMOR 3

Subset 1 - CLAHE
 Solusi Min -2%: 40x40 px, N=10 (Akurasi 60.83%)
 Solusi Min -5%: 30x30 px, N=10 (Akurasi 59.17%)

Subset 1 - Gamma
 Solusi Min -2%: 50x50 px, N=10 (Akurasi 83.33%)
 Solusi Min -5%: 30x30 px, N=10 (Akurasi 80.00%)

Subset 2 - CLAHE
 Solusi Min -2%: 50x50 px, N=10 (Akurasi 63.33%)
 Solusi Min -5%: 40x40 px, N=10 (Akurasi 60.00%)

Subset 2 - Gamma
 Solusi Min -2%: 50x50 px, N=10 (Akurasi 59.17%)
 Solusi Min -5%: 30x30 px, N=10 (Akurasi 55.83%)
```

*Gambar 8 - Output Kode Matlab Pengerjaan Nomor 3*

Terlihat pada grafik Gambar 7 bahwa jumlah minimum eigenface yang diperlukan untuk mempertahankan akurasi -2% dan -5% dari akurasi terbaik adalah pada N = 10. Hal ini dikarenakan setiap grafik untuk nilai N < 10 mengalami penurunan akurasi yang sangat signifikan. Pada Subset 1, resolusi citra minimum yang diperbolehkan untuk menjaga akurasi -2% bervariasi antara 40 x 40 px (metode CLAHE) hingga 50 x 50 px (metode Gamma). Sementara itu, resolusi minimum untuk menjaga akurasi -5% pada kedua metode adalah 30 x 30 px. Namun, ketika pencahayaan sangat buruk dan citra cukup terdistorsi seperti pada Subset 2, penurunan ukuran citra sangat berdampak pada stabilitas akurasi. Demi menjaga target akurasi -2% pada Subset 2, metode CLAHE dan Gamma mengharuskan penggunaan resolusi tetap di 50 x 50 px. Demi mencapai akurasi -5%, resolusi yang diperbolehkan adalah 40 x 40 px untuk CLAHE dan 30 x 30 px untuk Gamma. Oleh karena itu, pada kondisi ekstrem seperti Subset 2, penggunaan resolusi yang lebih tinggi (40 x 40 px atau 50 x 50 px) tetap menjadi pilihan yang paling disarankan untuk meminimalisir risiko kegagalan pengenalan wajah.

**Michael Liebing / 18323016**
**Jonathan Otto / 18323017**

**Nomor 4**

Cara menentukan tebakan awal nilai *threshold* yang baik adalah dengan memvisualisasikan *face space* sebagai bola berdimensi M yang mencakup seluruh vektor bobot dari seluruh basis data. Tebakan awal dapat diperkirakan sebagai jari-jari bola *face space* tersebut.

$$\theta_{threshold} = \frac{1}{2} max \sqrt{\left|\left|\Omega_{new} - \Omega_k\right|\right|^2}$$

Demi menilai keberadaan suatu citra dalam radius tersebut, dilakukan perhitungan selisih antara citra asli dengan hasil rekontruksinya menggunakan *N eigenface*. Jika citra tersebut dapat diproyeksikan cukup baik ke *face space*, selisihnya akan menjadi kecil.

$$\Phi_{recon} = \sum_{i=1}^{M} \omega_i \mu_i$$

$$\epsilon^2 = \left|\left|\Phi_{image} - \Phi_{recon}\right|\right|^2$$

$$\epsilon > \theta_{threshold}$$

Jika error rekonstruksi yang dihasilkan lebih besar dari *threshold*, gambar yang diuji kemungkinan besar bukanlah sebuah citra wajah.

**Nomor 5**

Berikut adalah kode fungsi `face_recognition.m` yang telah dimodifikasi untuk mengimplementasi pengenalan wajah melalui *threshold*,

```
function [acc,mindist,recog] = face_recognition_nomor5(w, labels, w2, labels2, N, th)
% w = training set, labels = training set labels
% w2 = test set, labels2 = test set labels
% N = number of eigenface components used (starting from the first
% eigenface corresponding to the highest eigenvalue)
% acc = accuracy (number of correct recognition/ total test set entries)
% mindist = minimum distance of a test set entry to a training set entry
% recog = recognition label, i.e the label of a training set entry which
% provides minimum distance to one test set entry
% th = threshold

if nargin < 6 || isempty(th)
    th = inf;
end

%% Initializations
v=w;                                % v contains the training set.
% N = Number of eigenface components used for each image (max(N) = number of images in the
training set)

%% Subtracting the mean from v
O=single((ones(1,size(v,2))));
m=single((mean(v,2)));              % m is the mean of all images.
vzm=v-(m*O);                        % vzm is v with the mean removed.
```

```
%% Calculating eigenvectors of the correlation matrix
% We are picking N eigenfaces corresponding to the first N largest eigen values.
L=single(vzm)'*single(vzm);
[V,D]=eig(L);
V=single(vzm)*V;
V=V(:,end:-1:end-(N-1));             % Pick the eigenvectors corresponding to the N largest
eigenvalues.

%% Calculating the signature weight for each image
cv = zeros(size(v, 2), N);
for i = 1:size(v, 2)
    cv(i,:) = single(vzm(:,i))' * V; % Each row in cv is the signature for one image.
end

%% Recognition
recog = [];
dist = [];
mindist = [];
for j = 1:size(w2, 2)
    r = w2(:, j);
    p = r - m;
    s = single(p)' * V;

    z = [];
    for i = 1:size(v, 2)
        z = [z, norm(cv(i,:) - s, 2)];
    end
    dist = [dist; z];


    valid_candidates = find(z <= th);

    if isempty(valid_candidates)
        recog = [recog, -1];

        mindist = [mindist, min(z)];
    else
        valid_distances = z(valid_candidates);

        [min_val, temp_idx] = min(valid_distances);

        best_match_idx = valid_candidates(temp_idx);

        recog = [recog, labels(best_match_idx)];
        mindist = [mindist, min_val];
    end
end

result = zeros(1, length(recog));
for j = 1:length(recog)
    result(j) = isequal(recog(j), labels2(j)); % result=1 if recog match labels2, 0 otherwise
end

acc = sum(result) / length(result); %accuracy

end
```

*Gambar 9 - Kode Matlab Fungsi face_recognition_nomor5.m*

Perubahan dalam kode Matlab fungsi *face_recognition.m* tersebut memastikan bahwa sistem tidak hanya mencari kemiripan terdekat, tetapi juga memvalidasi kelayakan kemiripan tersebut berdasarkan *threshold* yang ditentukan oleh pengguna. Bagian-bagian dari kode Matlab pada Gambar 9 yang dimodifikasi dari fungsi *face_recognition.m* telah ditandai dengan *highlight* berwarna kuning. Langkah pertama dalam modifikasi tersebut adalah penambahan argumen input `th` pada definisi fungsi dan logika pengondisian utnuk memberikan nilai *default* `inf` jika pengguna tidak menentukan nilai ambang batas secara spesifik. Hal tersebut memastikan bahwa kode tetap memiliki fleksibilitas tinggi dan dapat berfungsi seperti algoritma pencarian jarak minimum standar tanpa merusak program Matlab yang sudah ada. Langkah kedua adalah mengganti logika penentuan label langsung dengan proses filtrasi jarak Euclidean menggunakan fungsi `find(z <= th)`. Bagian dari kode Matlab tersebut berfungsi untuk mengisolasi indeks data di dalam basis data yang memiliki jarak lebih kecil atau sama dengan nilai ambang batas *threshold* yang telah ditentukan oleh pengguna. Langkah ketiga berkaitan dengan penanganan situasi tidak ada subjek yang memenuhi kriteria *threshold* tersebut. Jika variabel `valid_candidates` menghasilkan himpunan kosong, sistem akan memberikan label `-1` pada varaibel recog untuk menandakan bahwa citra tersebut tidak dikenal. Langkah terakhir adalah menerapkan mekanisme mitigasi untuk situasi *multiple entries*, yaitu ketika terdapat lebih dari satu entri dalam dataset yang lolos dari penyaringan *threshold*. Jika terjadi kasus tersebut, kode Matlab tersebut akan melakukan pemilihan ulang dengan mencari nilai jarak paling minimum hanya dari sekumpulan kandidat yang sudah dinyatakan valid.

**Nomor 6**

Berikut adalah kode Matlab fungsi `FARFRREER.m` yang digunakan untuk menghasilkan FAR/FRR/EER,

```
function [FAR, FRR, EER] = FARFRREER(sama, beda)
% Input Arguments:
% sama = jarak euclid orang yang sama
% beda = jarak euclid orang yang beda

% Output Arguments:
% FAR  = False Acceptance Rate
% FRR  = False Rejection Rate
% EER  = Equal Error Rate

    min_th = min([sama(:); beda(:)]);
    max_th = max([sama(:); beda(:)]);
    num_steps = 1000;
    thresholds = linspace(min_th, max_th, num_steps);

    FAR = zeros(1, num_steps);
    FRR = zeros(1, num_steps);

    total_gen = length(sama);
    total_imp = length(beda);

    for i = 1:num_steps
        th = thresholds(i);
```

```
        FRR(i) = sum(sama > th) / total_gen * 100;

        FAR(i) = sum(beda <= th) / total_imp * 100;
    end

    diff = abs(FAR - FRR);
    [~, min_idx] = min(diff);

    EER = (FAR(min_idx) + FRR(min_idx)) / 2;
end
```

*Gambar 10 - Kode Matlab Fungsi FARFRREER.m*

Kode Matlab fungsi tersebut bekerja dengan menganalisis distribusi jarak Euclidean dari dua kelompok input, yaitu kelompok sama yang berisi jarak antar wajah dari individu yang identik dan kelompok beda yang berisi jarak antar wajah dari individu yang berbeda. Langkah pertama dalam fungsi ini adalah menentukan rentang *threshold* pengujian dengan mencari nilai minimum dan maksimum dari seluruh data jarak, kemudian membaginya menjadi 1000 titik iterasi menggunakan perintah `linspace`. Dalam setiap iterasi, sistem menghitung FRR berdasarkan persentasi pasangan wajah asli yang salah ditolak karena jaraknya melebihi *threshold* dan menghitung FAR dari persentase pasangan wajah berbeda yang salah diterima karena jaraknya berada di bawah atau sama dengan *threshold*. Terakhir, nilai EER ditentukan dengan mencari titik dengan selisih absolut antara FAR dan FRR mencapai nilai paling minimum, lalu mengambil nilai rata-rata dari keduanya pada indeks tersebut sebagai indikator *threshold* optimal dengan tingkat kesalahan penolakan dan penerimaan pada posisi seimbang.